

`__init__()` method itself creates “surfaces” (images) for the button that are used in displaying a standard button. The code to create a typical `TextButton` object looks like this:

```
oButton = pygwidgets.TextButton(window, (50, 50), 'Text Button')
```

When drawn, the user sees a button that looks like [Figure 10-8](#).



Figure 10-8: An example of a typical `TextButton`

The `CustomButton` class is used to build a button using artwork the client supplies. Here is the interface for creating a `CustomButton`:

```
def __init__(self, window, loc, up, down=None, over=None,
             disabled=None, soundOnClick=None,
             nickname=None, enterToActivate=False):
```

The key difference is that this version of the `__init__()` method requires the caller to supply a value for the `up` parameter (remember, a button has four images: up, down, disabled, and over). You can optionally also supply down, over, and disabled images. For any image that is not supplied, `CustomButton` makes a copy of the up image of the button and uses that.

The last line of the `__init__()` methods for *both* the `TextButton` and `CustomButton` classes is a call to the `__init__()` method of the common base class, `PygWidgetsButton`. Both calls pass in four images for the button, along with other arguments:

```
super().__init__(window, loc, surfaceUp, surfaceOver,
                surfaceDown, surfaceDisabled, buttonRect,
                soundOnClick, nickname, enterToActivate, callBack)
```

From the client’s point of view, you see two completely different classes with many methods (most of which are identical). But from the implementer’s point of view, you can now see how inheritance allowed us to override the single `__init__()` method in the base class to provide client programmers with two similar, but very useful, ways of creating buttons.